Ledger App – HLD and LLD

Oct 31, 2026 · @tech

1. Scope and assumptions

This design covers a multi-shop ledger and billing system for saree and silk businesses, delivered as one codebase for mobile and web, with a shop-verification admin portal. The eight open points in the spec are closed below with defaults; each is a config or schema choice that is cheap to change later.

Open point	Decision used in this design
Circular swipe	Wraps both ways: Party → Sales → Purchases → Party. Tab buttons remain as a fallback.
Cheque dates	Cheque date, deposit date, clearing date. Deposit and clearing may stay empty until they happen.
Alert options	Per-entry reminder with a due date and a lead time in days (0, 1, 3, 7). Delivered as push and in-app.
Default GST %	Stored per shop, copied onto each bill as a snapshot, and editable on a single bill.
CGST+SGST vs IGST	Chosen automatically: same state as the shop (first two digits of both GSTINs) gives CGST+SGST at half rate each; otherwise IGST. Manual override allowed.
Artisans	Third party type: Customer, Supplier, Artisan. Artisan payments feed the Artisan and Job Work Ledger.
Return example	Yes, 100 sarees on a 200,000 bill gives a 2,000 per-saree gross. The system stores per-line quantities, so proration is exact, not inferred.
Admin verification	Phase 1: GSTIN format and checksum plus admin review of the entered details. Phase 2: GST portal lookup and document upload. Admin accounts are created by a seed script, never by self-signup.

Open point	Decision used in this design
Notifications	Registration, approval or rejection, plus due-date reminders (reusing the alert option).

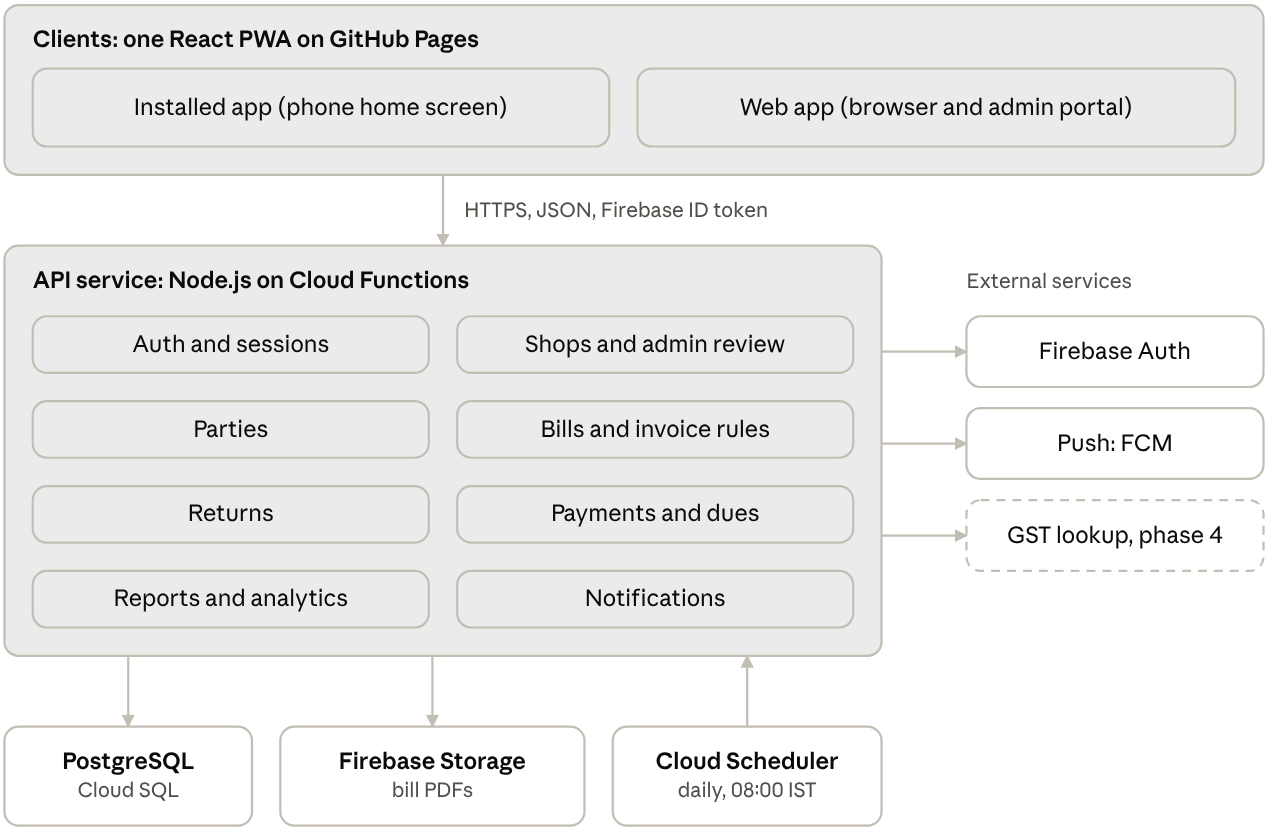
Assumptions

- Login and registration run on Firebase Authentication (Google, email and password, password reset). The shop-registration step and the approval gate are new.
- One user belongs to one shop in v1. Staff roles and multi-user shops are out of scope.
- Currency is INR, amounts are stored as integer paise, and dates are stored as UTC with display in Asia/Kolkata.
- Online-first. Offline entry with sync is a later phase.

2. HLD: Architecture

An installable React app (PWA) talks to one stateless Node.js API in front of one PostgreSQL database. Firebase supplies login, file storage, push messaging and the serverless runtime for the API; the React build is hosted on GitHub Pages.

An installable React app, one Node API, one SQL database



architecture · 3 layers, 3 external services

The apps never touch the database. Every rule (approval gate, invoice uniqueness, tax split, returns) runs in the API, and the same pure rule functions are shared with the client for live previews.

Technology choices

Layer	Choice	Reason
Client	React.js (Vite) with Tailwind CSS, built as a PWA	One codebase for phone and desktop; installs from the browser like an app
Client hosting	GitHub Pages, deployed by GitHub Actions	Free static hosting tied to the repository
Auth	Firebase Authentication: Google, email and password, password reset	Replaces the old login; the API verifies the Firebase ID token

Layer	Choice	Reason
API	Node.js (Express, TypeScript) on Firebase Cloud Functions, 2nd generation	Serverless; shares the domain rules package with the client
Database	PostgreSQL on Cloud SQL	Transactions, partial unique indexes, row-level security, SQL aggregates for GST reports
Files	Firebase Storage	Generated bill PDFs; verification documents in phase 4
Jobs	Cloud Scheduler calling a function daily	Due reminders
Notifications	In-app inbox in SQL plus Firebase Cloud Messaging web push	Registration, approval, due reminders
Version control	Git and GitHub with pull requests and Actions	See section 9

Why SQL behind Firebase. Firestore is the usual Firebase choice, but this ledger needs multi-row transactions, a unique sale invoice number per shop, and GST totals grouped and summed across many bills. Those are natural in SQL. Firebase still supplies everything around the database. A Firestore-only variant is possible (invoice number as the document id inside a transaction, totals through aggregation queries) but would mean reworking section 4.

Cost note. Cloud Functions and Cloud SQL need the Firebase Blaze (pay-as-you-go) plan, and a Cloud SQL instance has a monthly running cost even when idle. Check current Firebase and Cloud SQL pricing before choosing the instance size.

Multi-tenancy. A shop is the tenant. `shop_id` comes from the token on every request and is enforced in queries and by row-level security. Admin users have no shop and reach only the `/admin` routes.

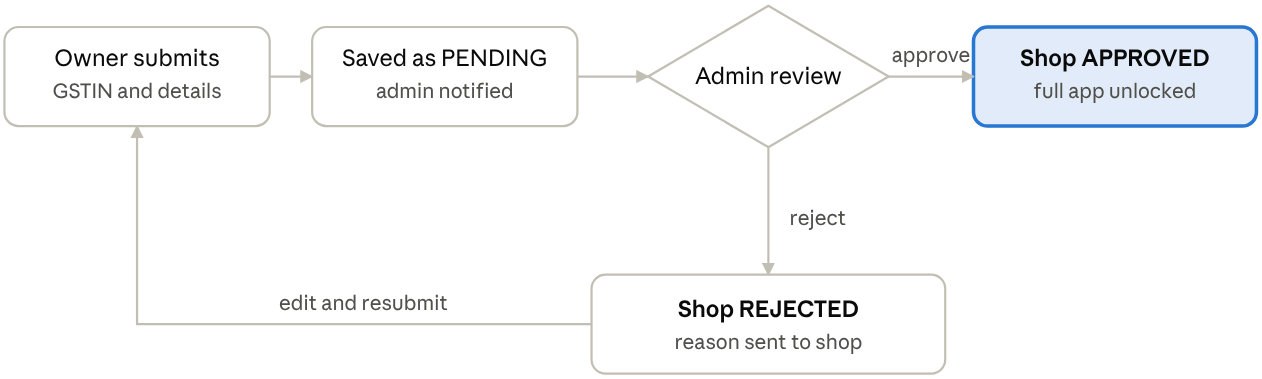
Approval gate. The API returns 403 on ledger, dues and analytics routes until the shop status is APPROVED, so the rule holds even if a client is modified.

3. HLD: Modules and key flows

The API is a modular monolith with one module per spec area: auth, shops and admin, parties, bills, returns, payments and dues, reports, notifications. One deployable keeps transactions simple (a return and its status refresh commit together); modules talk through function calls, not network hops, and can be split out later if a shop base grows.

Shop registration and approval

A shop reaches the ledger only after the admin approves it



shop verification flow · 1 decision, 1 loop back

A rejected shop sees the admin's reason on the pending-approval screen and can correct its details and resubmit. Each round is kept in `shop_verifications`, so the history is auditable.

Create a sale or purchase

1. The client validates fields, previews totals, and sends `POST /bills`.
2. In one transaction the API normalizes the invoice number, loads the shop and party state codes, computes the tax split, inserts the bill and its items, and writes an audit row.
3. A duplicate sale number violates the partial unique index; the API answers 409 with the next free number. Purchases skip this check.
4. The bill PDF is generated, stored, and its link returned. If a reminder date was set, it is saved on the bill for the daily job.

Record a return

1. From History the user opens Return sarees, picks the bill and enters a quantity per line; the client previews the reduction with GST.

2. `POST /bills/{id}/returns` checks that each quantity fits what is left on the line.
3. The API inserts the return rows, refreshes the bill status, and writes the audit row. The History and Dues screens then show the reduced amount.

Collect or pay a due

1. On Dues the user taps Collect (customer) or Pay (supplier or artisan) on a row.
2. A sheet asks for mode details: cash (date, collected by), cheque (three dates), NEFT (UTR, date).
3. `POST /bills/{id}/payments` rejects an amount above what is due, stores the payment and its mode row, and refreshes the status. A bill reaching zero moves from Pending to Completed.

Notifications

Event	Recipient	Channel
New shop registered	Admin	In-app and push
Shop approved or rejected	Shop owner	In-app and push, with the reason on rejection
Reminder date reached	Shop owner	In-app and push, from the daily job

4. LLD: Data model

PostgreSQL, one database, every business table carries `shop_id` for tenant isolation. Money is stored as integer paise (`BIGINT`); GST rates as basis points (1800 = 18%). Rows use UUID primary keys and `created_at` , `updated_at` , and soft-delete `deleted_at` where history matters.

Tenancy and access

Table	Key columns	Constraints and notes
shops	id, gstin, name, phone, email, state_code, status (PENDING, APPROVED, REJECTED), default_gst_bp, verified_at, verified_by	gstin unique, 15 chars matching GSTIN pattern; state_code derived from the first two digits

Table	Key columns	Constraints and notes
users	id, shop_id (null for admin), firebase_uid, email, role (OWNER, ADMIN)	firebase_uid and email unique; one shop per user in v1
shop_verifications	id, shop_id, submitted_details (JSON), decision, reason, decided_by, decided_at	One row per review round; rejected shops can resubmit
notifications	id, user_id, kind, payload (JSON), read_at	Kinds: SHOP_REGISTERED, SHOP_APPROVED, SHOP_REJECTED, DUE_REMINDER
device_tokens	id, user_id, platform, token	For push delivery

Parties

Table	Key columns	Constraints and notes
parties	id, shop_id, name, type (CUSTOMER, SUPPLIER, ARTISAN), owner_name, gstin, state_code, address, joined_on	Index on (shop_id, name) for search; joined_on drives the calendar dots
party_phones	id, party_id, phone, position	At least two at creation; more allowed

Bills, items and returns

Table	Key columns	Constraints and notes
bills	id, shop_id, party_id, kind (SALE, PURCHASE, JOB_WORK), invoice_no (INT), bill_date, gst_bp, tax_mode (INTRA, INTER), subtotal, cgst, sgst, igst, grand_total, status (PENDING, SETTLED), remind_on, remind_lead_days	Partial unique index on (shop_id, invoice_no) where kind = SALE. Purchases and job work have no uniqueness. grand_total is the original amount and never changes after returns.

Table	Key columns	Constraints and notes
bill_items	id, bill_id, name, qty, rate, line_total	line_total = qty × rate, validated server-side
returns	id, shop_id, bill_id, returned_on, taxable_reduction, cgst, sgst, igst, total_reduction	One return event per submission
return_items	id, return_id, bill_item_id, qty	Sum of returned qty per bill_item can never exceed bill_items.qty

Money movement

Table	Key columns	Constraints and notes
payments	id, shop_id, bill_id, amount, mode (CASH, CHEQUE, NEFT), paid_on	Direction follows the bill: Collect for SALE, Pay for PURCHASE and JOB_WORK
payment_cash	payment_id, collected_by	Cash details
payment_cheque	payment_id, cheque_date, deposit_date, clearing_date	Cheque details
payment_neft	payment_id, utr_no, txn_date	NEFT details; utr_no unique per shop
audit_log	id, shop_id, actor_id, entity, entity_id, action, before, after, at	Written for every edit, delete and return

Derived values, not stored

Amount due on a bill = grand_total minus total_reduction of its returns minus the sum of its payments. A bill is SETTLED when this reaches zero; the status column is a cache refreshed in the same transaction as any payment or return. Dashboard and analytics totals are computed from these tables with SQL aggregates, and a materialized view can be added if a shop outgrows live queries.

Why one `bills` table. Sales, purchases and job work share the same shape (party, items, tax, payments, returns). A `kind` column keeps the invoice, return and dues code in one path and makes the three ledger views simple filters. The only rule that differs, invoice uniqueness, is a partial index.

5. LLD: API design

REST over HTTPS, JSON bodies, versioned under `/api/v1`. Auth is a Firebase ID token sent as a bearer token and verified with the Firebase Admin SDK on every request. Every request is scoped to the caller's `shop_id` taken from the token, never from the body. List endpoints take `q`, `page` and `limit`; errors use one shape: `{ code, message, fields? }`.

Area	Endpoint	Purpose
Auth	POST /auth/register-shop	After the client creates the Firebase account, create the user and shop rows in PENDING state; notify admin
Auth	Firebase Authentication SDK (no API endpoint)	Email and password sign-in, Google sign-in and password reset run in Firebase
Admin	GET /admin/shops?status=PENDING	Review queue
Admin	POST /admin/shops/{id}/decision	Body: approve or reject, reason; notifies the shop
Parties	GET, POST /parties	Search and create
Parties	DELETE /parties/{id}	Blocked if the party has open dues; otherwise soft delete
Parties	GET /parties/join-dates?month=	Dates for the calendar dots
Bills	POST /bills	Create a sale, purchase or job-work bill; returns the bill and its PDF link
Bills	GET /bills?kind=&q=&status=	History list and ledger views
Bills	PATCH /bills/{id}	Edit item or date; totals recomputed; audited
Bills	DELETE /bills/{id}	Soft delete; blocked if payments exist unless forced by owner
Bills	GET /bills/next-invoice-no?kind=SALE	Suggested next number

Area	Endpoint	Purpose
Returns	POST /bills/{id}/returns	Body: items with qty; returns the new amounts
Returns	GET /returns	Returns view in History
Dues	GET /dues?side=PENDING or COMPLETED&q=	Table rows: party, invoice, date, amount due, status
Dues	POST /bills/{id}/payments	Body depends on mode; recomputes status
Share	POST /bills/share	One, many or all old transactions as PDF or link
Analytics	GET /analytics/summary	Totals, pending, settled, net balance, sales vs purchases
Reports	GET /reports/{purchase-ledger, sales-ledger, job-work-ledger, gst-dashboard}?from=&to=	The four sheets
Notifications	GET /notifications, POST /notifications/{id}/read	In-app inbox

Validation rules (enforced on the server, mirrored on the client)

Field	Rule
GSTIN	15 characters matching `^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]$`, plus the mod-36 check character
Phone	10 digits, optional +91, normalized on save
Password	Firebase Auth password policy (minimum 8); confirm password must match, checked on the client only
Invoice number	Digits only; strip leading zeros (`01` becomes 1, `00100` becomes 100); reject empty or zero; for SALE, reject a duplicate within the shop with HTTP 409

Field	Rule
Items	At least one line; qty > 0; rate ⩾ 0; line_total must equal qty × rate
Return qty	Per item, qty ⩽ original qty minus already returned
Payment	Amount ⩽ amount due; mode-specific fields required (cash: date and collector; cheque: three dates; NEFT: UTR and date)

Example: create sale

```
POST /api/v1/bills
{
  "kind": "SALE",
  "party_id": "7b1c...",
  "invoice_no": "0042",
  "bill_date": "2026-10-31",
  "gst_bp": 500,
  "items": [
    { "name": "Kanchipuram silk saree", "qty": 10, "rate": 1250000 }
  ],
  "remind_on": "2026-11-30",
  "remind_lead_days": 3
}
```

The response returns `invoice_no: 42`, the tax split chosen from the two state codes, and the computed totals, all in paise.

6. LLD: Business logic

All money math runs on the server in integer paise, inside a single database transaction with the write it supports. The client shows a live preview using the same pure functions, packaged once in a shared TypeScript module.

Invoice number normalization

```
normalize_invoice(raw):
    if raw contains any non-digit: reject
    n = raw with leading zeros removed
    if n is empty: reject
    return integer(n)
```

For a SALE the unique partial index is the final guard. On a conflict the API returns 409 with the next free number, so a race between two devices cannot save a duplicate.

Tax split

```
compute_tax(items, gst_bp, shop_state, party_state):
    subtotal = sum(item.qty * item.rate)
    tax      = round_half_up(subtotal * gst_bp / 10000)
    if shop_state == party_state:
        sgst = floor(tax / 2);  cgst = tax - sgst;  igst = 0
    else:
        cgst = 0;  sgst = 0;  igst = tax
    return subtotal, cgst, sgst, igst, subtotal + tax
```

The rate is copied onto the bill when it is saved, so later changes to the shop default never alter old bills. A party without a GSTIN (an unregistered retail customer) takes the shop's state, so the split is CGST + SGST.

Returns: proportional reduction

The reduction is computed per line from the bill's own rate, so the result matches the spec's example exactly: 50 of 100 sarees on a 200,000 bill with GST removes 100,000 with GST.

```
record_return(bill, returned):          # returned = {bill_item_id: qty}
    for each (item, qty) in returned:
        assert qty > 0 and qty <= item.qty - already_returned(item)
    taxable = sum(qty * item.rate)
    tax     = round_half_up(taxable * bill.gst_bp / 10000)
    split tax into cgst/sgst/igst using the bill's tax_mode
    insert returns + return_items (taxable, cgst, sgst, igst, taxable + tax)
    refresh_status(bill)
    write audit_log
```

The bill's original grand_total is never overwritten. The displayed bill amount is original minus returns, which keeps the audit trail clean and makes a return reversible by deleting the return row.

Dues and status

```
amount_due(bill) = bill.grand_total - sum(returns.total) -
sum(payments.amount)
refresh_status(bill): status = SETTLED if amount_due == 0 else PENDING
```

- A return can push amount_due to zero or below. If it goes below zero (the party already paid more than the reduced bill), the excess is shown as a credit on the party card and is not auto-refunded.
- The Dues screen lists bills with amount_due > 0 under Pending and amount_due = 0 under Completed. The action button reads Collect for SALE and Pay for PURCHASE and JOB_WORK.
- Deleting a bill that has payments is blocked by default, because it would silently erase money records.

Reports

Sheet	Computation
Purchase Ledger	PURCHASE bills in range; taxable value, CGST, SGST, IGST per bill; ITC = sum of the three, net of returns, only for suppliers with a GSTIN
Sales Ledger	SALE bills; retail counter (party without GSTIN or same state) vs out-of-state (IGST); payment status from dues
Artisan and Job Work	JOB_WORK bills by artisan; paid, outstanding, GST
GST Dashboard	Output tax = CGST + SGST + IGST on sales; ITC from purchases; net payable = output − ITC; monthly sales vs purchase series

Reminders. A daily job at 08:00 IST finds bills where `remind_on - remind_lead_days` is today and amount_due > 0, then creates a DUE_REMINDER notification and a push per bill. It is idempotent: a reminder is keyed by bill and date so a retry never double-sends.

7. LLD: Client design

One React.js codebase (Vite, TypeScript) styled with Tailwind CSS runs in any browser and installs to a phone or desktop as a PWA, so every screen is identical as the spec requires. Server state is handled by TanStack Query (cache, retries, refetch after payments and returns); local UI state by component state and a small Zustand store for the session and default GST.

Installable app (PWA)

- A web app manifest (name, 192 and 512 px icons including a maskable one, theme colour, standalone display, start URL) and a service worker from `vite-plugin-pwa` make the app installable: an install prompt in Chrome and Edge, Add to Home Screen in Safari on iPhone.
- The service worker caches the app shell so the app opens instantly and shows a clear offline message. API calls are network-first and never cached, so ledger numbers are not stale.
- Tailwind is used mobile-first: one column with bottom navigation under 768 px, side navigation and wider tables above it. Touch targets are at least 44 px.
- Push uses Firebase Cloud Messaging web push through the same service worker. On iPhone it works only after the app is installed to the home screen (iOS 16.4 or later).
- Updates: the app checks for a new build on launch and shows an Update available prompt. Each GitHub Pages deploy replaces the previous build.
- GitHub Pages serves the site from a repository sub-path and cannot rewrite single-page-app routes. Set the Vite `base` to the repository name and use hash routes (`/#/dues`), or add a 404.html redirect.

Navigation

Route	Screen	Gate
/login, /register-shop, /forgot-password, /admin-login	Auth screens	Public
/pending-approval	Waiting or rejected message with the reason	Logged in, shop not APPROVED
/dashboard	Four cards linking to the four report sheets	Approved shop

Route	Screen	Gate
/parties	Search, Add new member, join-date calendar, party cards	Approved shop
/ledger	Pager with Party, Sales, Purchases views and the entry form	Approved shop
/history	History and Returns toggle, search, edit, delete, Return sarees	Approved shop
/dues	Pending and Completed tabs, table, Pay or Collect	Approved shop
/analytics	Summary tiles and sales vs purchases chart	Approved shop
/admin	Review queue with approve and reject	Admin role

A route guard reads the shop status from the session. Unapproved shops are redirected to /pending-approval, so ledger, dues and analytics are unreachable even by typing a URL; the API enforces the same rule independently.

Circular swipe on the Ledger screen

The three views live in a pager with a virtual index. A horizontal swipe changes the index by plus or minus one, and the visible view is `views[((index % 3) + 3) % 3]`, so swiping past Purchases lands on Party and swiping back from Party lands on Purchases. A threshold of about 25% of screen width or a fast fling commits the swipe, otherwise it springs back. The three tab buttons stay visible for accessibility and for desktop browsers, where swipe is replaced by arrow keys and tab clicks.

Ledger entry form behavior

- Party search is the only search field; picking a party fills its state code, which drives the CGST+SGST or IGST preview.
- The invoice field accepts digits only (input is filtered, paste is sanitized) and strips leading zeros on blur. For Sales it checks availability against the server as the user types, debounced at 400 ms.

- Each item row computes its total on change; Add another item appends a row. Totals, the three tax lines and the grand total recompute from the shared pure functions.
- Create bill saves, then opens the generated bill PDF with share actions.

Dues payment sheet. Choosing Pay or Collect opens a bottom sheet with a mode selector. The form below it swaps by mode: cash (date, collected by), cheque (cheque, deposit and clearing dates), NEFT (UTR, transaction date). Validation is a discriminated union keyed by mode, shared with the server.

Suggested folder layout

```
ledger-app/                 # one Git repository (npm workspaces)
  web/                      # React + Vite + Tailwind PWA
    src/pages/              # Login, RegisterShop, Dashboard, Parties,
Ledger,
                            # History, Dues, Analytics, Admin
    src/components/         # cards, tables, date picker, pager, pay sheet
    src/lib/                # firebase.ts, API client, query hooks
    public/                 # manifest, icons
    tailwind.config.js
    vite.config.ts          # base path + vite-plugin-pwa
  functions/                # Node.js API on Cloud Functions
    src/modules/            # auth, shops, parties, bills, returns, payments,
                            # reports, notifications, admin
    src/jobs/               # daily reminder function
    src/db/                 # SQL migrations and queries
  shared/
    domain/                 # pure TS: tax, returns, invoice normalize,
validators
  .github/workflows/        # ci.yml, deploy-web.yml, deploy-api.yml
  firebase.json  .firebaserc
```

Sharing the `domain` package between client and server is the main guard against preview and saved totals disagreeing.

8. Security, quality and rollout

Security and compliance

- Tenant isolation: every query is filtered by `shop_id` from the token; PostgreSQL row-level security is enabled as a second layer so a missed filter cannot leak another shop's data.

- Passwords and sessions are handled by Firebase Authentication. The API verifies every ID token with the Admin SDK and checks token revocation on admin routes. Firebase App Check limits API calls to the real app.
- Admin routes require an admin custom claim on the Firebase user, rate limiting, and a separate login page. The claim is set by a one-time script run by the project owner; there is no admin self-signup.
- TLS everywhere, encrypted database storage, daily backups with 30-day retention and a tested restore.
- Audit log for every edit, delete, return and payment. Bills with payments cannot be hard-deleted; financial records are kept for the statutory period (GST records: at least 6 years, confirm with the shop's accountant).
- Personal data (phone numbers, addresses) is limited to what the spec collects and is deleted with the party only when no bills reference it.

Performance targets

Metric	Target
List and search endpoints	p95 under 300 ms for 50,000 bills per shop
Create bill, record payment, record return	p95 under 500 ms
Report sheets	Under 2 s for one financial year
Indexes	bills(shop_id, kind, bill_date), bills(shop_id, party_id), parties(shop_id, name), payments(bill_id)

Testing

- Unit tests on the shared `domain` package: invoice normalization, tax split on odd paise, return proration including the 200,000 / 100 / 50 example.
- API integration tests against a real PostgreSQL: duplicate sale invoice returns 409 under concurrent requests, purchases may repeat, return qty cannot exceed the bill, an unapproved shop gets 403 on ledger, dues and analytics.
- End-to-end tests for register shop, admin approve, create bill, return, pay by each mode.

Rollout phases

Phase	Scope	Exit check
1. Core	Shop registration and approval, parties, sales and purchase ledger, invoice rules, History	A pilot shop can enter a week of real bills
2. Money	Returns, Dues with the three payment modes, notifications and reminders	Dues match the shop's manual register
3. Reports	Analytics and the four GST and ledger sheets, artisan and job work	Accountant signs off the GST dashboard against a filed return
4. Hardening	GST portal lookup, document upload for verification, offline entry with sync	Load test and security review pass

Risks

Risk	Mitigation
Tax split wrong for unregistered or composition-scheme parties	Manual override per bill; accountant review in phase 3
Rounding differences against the shop's existing books	Round at the document level, not per line; document the rule
Two devices claim the same sale invoice number	Database partial unique index and the 409 response with the next number
Fake shop registrations	GSTIN checksum, admin review, and a GST portal lookup in phase 4
Circular swipe conflicts with horizontal scrolling inside tables	Swipe is only recognized outside scrollable regions; tab buttons always work

9. Deployment, version control and installation

Every change goes through Git and GitHub: pull requests run tests, a merge to `main` deploys the React app to GitHub Pages and the Node.js API to Firebase. GitHub Pages hosts only static files, so the API cannot live there; it runs on Firebase Cloud Functions

and the app calls it over HTTPS.

Git workflow

- One repository with npm workspaces (`web`, `functions`, `shared`), so client, API and shared rules are versioned together.
- `main` is protected: changes arrive by pull request from short-lived `feature/<name>` branches, with CI passing before merge.
- Commit messages follow Conventional Commits (`feat:`, `fix:`, `chore:`). Releases are tagged `v1.0.0`, `v1.1.0` and so on.
- SQL changes are numbered, forward-only migration files in `functions/src/db/migrations`, reviewed in the same pull request as the code that needs them.
- `.gitignore` excludes `.env` files, service-account JSON and `node_modules`. Nothing secret is committed.

Environments

Environment	Firebase project	Hosting	Deploy trigger
Development	ledger-dev	Local Vite server and Firebase emulators	Manual
Production	ledger-prod	GitHub Pages and Cloud Functions	Merge to `main`

The Firebase project aliases live in `.firebaserc`. A staging project can be added later without changing the workflow shape.

GitHub Actions workflows

File	Runs on	Steps
ci.yml	Every pull request	Install, lint, unit tests for `shared/domain`, API tests against a PostgreSQL service container, build web and functions
deploy-web.yml	Push to `main` touching `web` or `shared`	Build the PWA, publish `web/dist` to GitHub Pages

File	Runs on	Steps
deploy-api.yml	Push to `main` touching `functions` or `shared`	Run pending SQL migrations, deploy functions with the Firebase CLI

```
name: Deploy web
on:
  push:
    branches: [main]
    paths: [web/**, shared/**]
permissions:
  contents: read
  pages: write
  id-token: write
concurrency:
  group: pages
  cancel-in-progress: true
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run build --workspace web
        env:
          VITE_API_URL: ${{ vars.VITE_API_URL }}
          VITE_FIREBASE_API_KEY: ${{ vars.VITE_FIREBASE_API_KEY }}
          VITE_FIREBASE_AUTH_DOMAIN: ${{ vars.VITE_FIREBASE_AUTH_DOMAIN }}
          VITE_FIREBASE_PROJECT_ID: ${{ vars.VITE_FIREBASE_PROJECT_ID }}
      - uses: actions/upload-pages-artifact@v3
        with:
          path: web/dist
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
```

```
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

The Firebase web config values (API key, auth domain, project id) identify the project and are not secrets, so they are stored as repository variables. Real secrets stay out of the browser build: the service-account JSON used by `deploy-api.yml` is a GitHub encrypted secret, and the database password lives in Google Secret Manager and is read by the function at runtime.

One-time setup

- [ ] Create the Firebase projects, enable the Email/Password and Google sign-in providers, and add the GitHub Pages domain under Authentication authorized domains.
- [ ] Switch the project to the Blaze plan, create the Cloud SQL PostgreSQL instance, and connect the functions to it.
- [ ] Enable Firebase Storage, Cloud Messaging and App Check.
- [ ] Create the GitHub repository, set Pages source to GitHub Actions, and add the repository variables and the service-account secret.
- [ ] Restrict the API's CORS to the Pages origin.
- [ ] Run the admin-claim script once for the first admin user.
- [ ] Protect `main` and require the CI checks.

How users install the app

Device	Steps
Android (Chrome)	Open the site, tap Install app in the prompt or the browser menu
iPhone (Safari)	Tap Share, then Add to Home Screen
Desktop (Chrome, Edge)	Click the install icon in the address bar

The installed app opens full screen with its own icon, receives push notifications, and updates itself from the latest GitHub Pages build.